

NGS2026 — Report Tecnico

Benchmark Plate Detection (Hailo-8) e OCR (EasyOCR vs Gemini API)

Maggio 2026 — Edge AI Pipeline for Smart Cities & Parking

Sintesi

Questo report documenta due benchmark eseguiti per migliorare la pipeline di vehicle detection e lettura targhe del progetto NGS2026. I test sono stati condotti su Raspberry Pi 5 con Hailo-8 AI HAT+ per la plate detection, e con accesso a Gemini API via Google Cloud per il confronto OCR. Tutti i benchmark sono stati eseguiti su dataset estranei al training dei modelli per garantire imparzialità.

Principali risultati

- **Plate detection:** license_plate_finetune_v1n vince con $F1 = 0.9555$ (vs 0.849 di uc3m e 0.638 di tiny_yolov4). Diventa il nuovo candidato per sostituire tiny_yolov4 nella pipeline.
 - **OCR:** Gemini 2.5 Flash è il vincitore (81.2% exact match), molto superiore a EasyOCR (16.7%) su targhe europee miste. Pro non offre vantaggi rispetto a Flash.
 - **Strategia consigliata:** sostituire tiny_yolov4 con v1n; mantenere EasyOCR per targhe italiane in produzione, valutare Gemini Flash come fallback per casi di bassa confidenza.
-

1. Benchmark Plate Detection — Confronto a 3 modelli HEF

1.1 Setup

Modelli confrontati

- license_plate_finetune_v1n.hef — Federico, training su dataset V1N
- yolov11n_uc3m.hef — Federico, training su dataset UC3M
- tiny_yolov4_license_plates.hef — Hailo Model Zoo (modello attualmente in produzione)

Dataset di valutazione

Persian Car Plates (Roboflow), 1381 immagini con 1474 ground truth bounding box. Dataset completamente estraneo a tutti e 3 i training set: rappresenta un test di generalizzazione imparziale. Le label sono in formato misto YOLO detection + YOLO segmentation, gestite con conversione automatica al volo.

Metodologia

- Inferenza diretta su Hailo-8 via HailoRT (NMS integrato per i 2 modelli yolov11n, decoding manuale per tiny_yolov4)
- Matching IoU = 0.5 (standard COCO) per TP/FP/FN

- Letterboxing con preservazione aspect ratio
- Letterboxing inverso per riportare le predizioni in coord. originali

1.2 Risultati

Modello	F1-score	Precision	Recall	mAP@50	FPS
v1n	0.9555	0.917	0.998	0.994	108.8
uc3m	0.8490	0.749	0.980	0.956	91.1
tiny_yolov4	0.6377	0.615	0.662	0.602	447.8

Dettaglio TP / FP / FN su 1474 ground truth

Modello	True Pos.	False Pos.	False Neg.
v1n	1471	134	3
uc3m	1445	485	29
tiny_yolov4	976	611	498

1.3 Analisi

v1n — eccellente in ogni metrica

- Recall 99.8%: solo 3 targhe perse su 1474. Praticamente non sbaglia mai a rilevare una targa presente.
- Precision 91.7%: di tutto ciò che etichetta come targa, oltre il 91% lo è davvero.
- mAP@50 = 0.9942: indica una qualità di detection vicino al massimo teorico.

uc3m — buon detector ma rumoroso

- Recall analogo a v1n (98%), il modello "vede" quasi sempre la targa.
- Precision 74.9%: triplica i falsi positivi rispetto a v1n (485 vs 134). Tende a vedere targhe anche dove non ci sono (es. scritte sull'auto, riflessi, finestrini).

tiny_yolov4 — il più veloce, il meno accurato

- 4× più veloce di v1n, ma con accuratezza dimezzata.
- 498 FN (perde 1 targa su 3) e 611 FP. Non robusto su questo dataset.
- La velocità in eccesso (448 FPS) non è una leva utile: la pipeline NGS non ha bisogno di frame rate superiori a ~30 FPS.

Conclusione Plate Detection

v1n è il modello vincente in modo netto e statisticamente robusto. Si propone come sostituto di tiny_yolov4 nella pipeline NGS, con un guadagno di accuratezza significativo (F1 da 0.638 a 0.955) e una velocità ancora abbondantemente sopra le richieste operative.

2. Benchmark OCR — EasyOCR vs Gemini API

2.1 Setup

Modelli confrontati

- EasyOCR (en, allowlist A-Z + 0-9) — soluzione on-device usata nella pipeline attuale
- Gemini 2.5 Flash-Lite — modello low-cost di Google
- Gemini 2.5 Flash — modello veloce con thinking disabilitato
- Gemini 2.5 Pro — testato in pilota (escluso dal run completo perché non offre vantaggi su Flash)

Dataset di valutazione

European License Plates, 735 crop di targa di vari paesi europei (IT, FR, ES, DE, PT, IRL, ecc.). Ground truth derivata dal nome del file (es: "THW82986.png" → GT = "THW82986"). Le targhe sono in formati eterogenei: l'OCR deve generalizzare oltre il pattern italiano AA000AA.

Metodologia

- Per ogni crop, ogni modello restituisce il testo letto
- Exact Match: la predizione è uguale carattere per carattere alla GT
- CER (Character Error Rate): Levenshtein distance / lunghezza GT
- Confusion matrix carattere-per-carattere per identificare errori sistematici
- Gemini chiamato via Vertex AI con ADC, 5 chiamate concorrenti

2.2 Risultati

Modello	Exact Match	Accuracy	Avg CER	Tempo medio
Gemini 2.5 Flash	597 / 735	81.2%	0.055	0.98 s
Gemini 2.5 Flash-Lite	512 / 735	69.7%	0.078	1.23 s
EasyOCR	123 / 735	16.7%	0.698	1.05 s

Pilota Gemini Pro (50 immagini)

In un test pilota su 50 immagini, Gemini 2.5 Pro ha ottenuto risultati identici a Gemini 2.5 Flash: stesso numero di Exact Match (43/50, ovvero 86%), stesso CER medio (0.035) e perfino le stesse confusioni carattere per carattere. L'unica differenza significativa è il tempo: Pro ha impiegato in media 6.2 secondi

per immagine, ovvero 5× più lento di Flash. Poiché Pro non offre alcun vantaggio di accuratezza e costa circa 10× di più, è stato escluso dal run completo su 735 immagini. Il confronto definitivo è quindi tra Flash, Flash-Lite e EasyOCR.

2.3 Analisi degli errori

Errori di Gemini Flash — onesti e prevedibili

Carattere GT	Predetto	Occorrenze
0 (zero)	O (lettera)	17
I	1	6
Q	O	3
0	G	3

Le confusioni di Flash sono dominate da caratteri visivamente ambigui (0/O, I/1, Q/O). Su targhe italiane in formato AA000AA, queste confusioni sarebbero auto-correggibili tramite filtro posizionale (le posizioni 1-2 e 6-7 sono obbligatoriamente lettere, le posizioni 3-5 sono obbligatoriamente cifre).

Errori di EasyOCR — più grossolani

Carattere GT	Predetto	Occorrenze
0 (zero)	O (lettera)	43
4	L	19
7	Z	15
D	0	11
M	H	11
1	I	11
W	M	9

EasyOCR ha errori O/O analoghi a Flash, ma in più sbaglia su confusioni di forma del carattere (4→L, 7→Z, M→H, W→M). Questi errori indicano un riconoscimento OCR fragile su font o proporzioni di targhe non italiane.

Importante: il dataset è di targhe europee miste

EasyOCR è stato originariamente testato e validato dalla pipeline NGS su video di parcheggio italiano, con risultati buoni. Il dataset di questo benchmark contiene targhe da numerosi paesi europei (UK, Francia, Germania, Portogallo, ecc.) con formati e font molto eterogenei. È plausibile che EasyOCR su un dataset di sole targhe italiane abbia performance significativamente migliori, ma questo va verificato in un benchmark futuro.

Conclusione OCR

Gemini Flash è il vincitore univoco per accuratezza su targhe europee miste. Pro non aggiunge valore. Flash-Lite è competitivo come opzione low-cost. EasyOCR è inadatto a targhe estere senza fine-tuning, ma resta valido per targhe italiane in condizioni operative tipiche del progetto.

3. Raccomandazioni operative

3.1 Sostituzione plate detection

- Sostituire `tiny_yolov4` con `license_plate_finetune_v1n` nella pipeline produttiva.
- Validare `v1n` anche su un dataset di targhe italiane prima del rollout finale, per garantire che le metriche tengano sul caso d'uso reale.

3.2 Strategia OCR ibrida

- Mantenere EasyOCR come OCR primario on-device: zero costo, niente dipendenza da rete.
- Implementare validazione formato italiano post-OCR (AA000AA): le predizioni che non matchano vengono marcate come incerte.
- Per le incerte: opzionale fallback a Gemini Flash via API, con costo trascurabile (~0.001€ per chiamata) per recuperare i casi difficili.
- Implementare preprocessing pre-OCR sul crop targa (grayscale + CLAHE + adaptive threshold) per ridurre errori di EasyOCR su immagini in condizioni non ottimali.

3.3 Test rimanenti da pianificare

- Benchmark plate detection `v1n` su dataset di sole targhe italiane (numericamente robusto, almeno 500+ immagini).
- Confronto OCR EasyOCR vs Gemini Flash su targhe italiane: cambia la classifica?
- Benchmark vehicle detection (`yolov11s`) su dataset estraneo con vista dall'alto (es. UA-DETRAC, MIO-TCD).
- Test del preprocessing pre-OCR misurato in CER su entrambi i motori.
- Integrazione IMX500 (ad oggi sospesa).

Appendice tecnica

A. Note sul formato delle label

Il dataset Persian Car Plates contiene label in formato misto: alcune righe in formato YOLO detection (5 valori), altre in formato YOLO segmentation (poligoni a 4 vertici). Lo script di benchmark gestisce entrambi i casi: i poligoni vengono convertiti in bounding box axis-aligned tramite min/max delle coordinate.

B. Note su Gemini API

Le chiamate Gemini sono state effettuate via Vertex AI su Google Cloud con ADC (Application Default Credentials). Il "thinking mode" è stato disabilitato sui modelli Flash e Flash-Lite (impostando `thinking_budget=0`), che altrimenti generano chain-of-thought leakage nell'output. Gemini Pro non supporta `thinking_budget=0` ma non ne ha bisogno per produrre output puliti. Costo totale stimato del benchmark completo: meno di 1€.

C. Note sui tempi di inferenza

I tempi di EasyOCR e di Gemini sono nominalmente comparabili (~1 secondo) ma la natura è diversa: EasyOCR consuma CPU della Raspberry Pi, mentre Gemini consuma latenza di rete (round-trip al server). In condizioni di alta concorrenza o di rete scadente, Gemini può subire impatti che EasyOCR non ha. Per pipeline real-time critiche è quindi consigliato EasyOCR; per OCR offline o batch, Gemini Flash è preferibile.

— Fine del report —